
Automated Data Labeling for Autonomous Aircraft Perception System

UNDERGRADUATE THESIS

*Submitted in fulfillment of the requirements of
BITS F421T Thesis*

By

Mohammed Abdul Hafeez KHAN
ID No. 2019A7TS0091U

Under the supervision of:

Dr. Raja MUTHALAGU
Associate Professor
BITS Pilani, Dubai Campus



Birla Institute of Technology And Science Pilani, Dubai Campus,
Dubai, U.A.E

May 2023

Declaration of Authorship

I, Mohammed Abdul Hafeez KHAN, declare that this Undergraduate Thesis titled, ‘Automated Data Labeling for Autonomous Aircraft Perception System’ and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed:

Date:

Certificate

This is to certify that the thesis entitled, “*Automated Data Labeling for Autonomous Aircraft Perception System*” and submitted by Mohammed Abdul Hafeez KHAN ID No. 2019A7TS0091U in fulfillment of the requirements of BITS F421T Thesis embodies the work done by him under my supervision.

Supervisor

Dr. Raja MUTHALAGU
Associate Professor,
BITS Pilani, Dubai Campus,
United Arab Emirates
Date:

Co-Supervisor

Dr. Siddhartha BHATTACHARYYA
Associate Professor,
Florida Institute of Technology,
United States of America
Date:

Birla Institute of Technology And Science Pilani, Dubai Campus

Abstract

Bachelor of Engineering (Hons.)

Automated Data Labeling for Autonomous Aircraft Perception System

by Mohammed Abdul Hafeez KHAN

The usage of machine learning and deep learning for designing and deploying cutting-edge systems is exponentially rising, and vast volumes of data with suitable labels are necessary to create these models. In this research, the focus is on the development of autonomous aircraft perception system for accurately detecting and labeling the line markings on an airport taxiway. A generalized framework called an Automated Line Identification and Notation Algorithm (ALINA) has been proposed, which can be used for labeling any taxiway datasets that may consist of complex environmental settings, changeable weather, and atmospheric conditions. ALINA begins with an interactive process for specifying a trapezoidal region of interest (ROI), which is subsequently used across all images in the dataset. The ROI undergoes geometrical modification and color space transformation for generating a binary map of the pixels, which is then subjected to histogram analysis. A CIRCular threshoLd pixEl Discovery And Traversal (CIRCLEDAT) algorithm has been proposed as part of ALINA, which acts as an integral step in determining the pixels corresponding to line markings on the airport taxiway. The output generates annotations of the line markings on the image along with a text file of its corresponding coordinates. Using this approach, 60,249 images from the taxiway dataset have been labeled accurately. The detection rate obtained by ALINA after testing it with 120 ground truth images consisting of different scenarios of the airport taxiway was recorded as 0.9844, attesting its dependability and effectiveness.

Acknowledgements

I am grateful to have had the opportunity to work on my undergraduate thesis on "Automated Data Labeling for Autonomous Aircraft Perception System". Firstly, I would like to convey my sincere appreciation to Dr. Srinivasan Madapusi, the Director, for his unwavering guidance and support throughout my academic journey. I am thankful to Dr. K. Kumar, the Associate Dean of Academic Undergraduate Studies (AUGSD), for constantly encouraging me. I would also like to express my gratitude to Dr. Vincent S. Kumar from the International Programmes and Collaboration Division (IPCD) for his guidance and assistance.

My heartfelt thanks go to my supervisor, Dr. Raja Muthalagu, and my co-supervisor, Dr. Siddhartha Bhattacharyya, for their exceptional support and mentorship. Their guidance and motivation have played a pivotal role in shaping the trajectory of my research.

I would like to extend my sincere gratitude to Parth Ganeriwala, the Graduate Student Advisor (GSA), for his support and contributions to my research. Furthermore, I would like to express my heartfelt appreciation to Dr. Natasha Neogi, Research Engineer at National Aeronautics and Space Administration (NASA), for her invaluable insights and guidance towards my research work.

I would also like to thank my family and friends for their constant support and encouragement. Their love and care have been the driving force behind my accomplishments. Lastly, I would like to sincerely acknowledge all the individuals who have directly or indirectly contributed to my research work.

I am deeply grateful to everyone who has made my academic journey an incredibly enriching and memorable experience. Thank you all.

Signed:

Mohammed Abdul Hafeez KHAN

ID No. 2019A7TS0091U

Contents

Declaration of Authorship	i
Certificate	ii
Abstract	iii
Acknowledgements	iv
Contents	v
List of Figures	vii
List of Tables	viii
Abbreviations	ix
1 Introduction	1
1.1 Problem and Motivation	1
1.2 Objectives	2
1.3 Contributions	2
1.4 Outline	3
2 Literature Review	4
3 Research Background	9
3.1 OpenCV	10
3.1.1 Loading and Displaying Images	10
3.1.1.1 imread()	10
3.1.1.2 imshow()	11
3.1.1.3 imwrite()	11
3.1.1.4 waitKey()	11
3.1.1.5 destroyAllWindows()	12
3.1.2 Image Processing	12
3.1.2.1 cvtColor()	12
3.1.2.2 polylines()	12

3.1.2.3	<code>inRange()</code>	12
3.1.2.4	<code>bitwise_and()</code>	13
3.1.2.5	<code>split()</code>	13
3.1.2.6	<code>normalize()</code>	13
3.1.3	Geometric Transformations	14
3.1.3.1	<code>getPerspectiveTransform()</code>	14
3.1.3.2	<code>warpPerspective()</code>	14
3.2	NumPy	14
3.3	Matplotlib	15
3.4	Canny Edge Detection Algorithm	16
4	Methodology	17
4.1	Image Representation	17
4.2	Interactive ROI Definition for Image Analysis	18
4.3	Advanced Perspective Transformation of the ROI	19
4.4	Color Feature Normalization for Enhanced Image Processing	20
4.5	Color Thresholding for Image Segmentation	20
4.6	Analyzing Image Data Distribution with Histogram Plotting	21
4.7	Identifying Line Markings and Avoiding False Positives	22
4.8	CIRCLEDAT: CIRCular threshoLd pixEl Discovery and Traversal Algorithm	22
4.9	Image Unwarping and Marking the Line Markings Pixels	24
5	Experimental Results	26
5.1	ALINA Performance: Specifications and Scenarios	26
5.2	Ground Truth Generation	28
5.3	Detection Performance: ALINA vs Ground Truth	30
6	Conclusion	33
	Bibliography	34

List of Figures

4.1	Flowchart of ALINA	17
4.2	Image Instance from the Taxiway Dataset	18
4.3	ROI highlighted in red on the image	18
4.4	Bird's Eye View of the ROI	19
4.5	Color Feature Normalization using HSV components	20
4.6	Color Thresholding based on Hue Component	21
4.7	Histogram of white pixels across the x-axis in the image	22
4.8	Result of Applying CIRCLEDAT on the Thresholded Image	23
4.9	Inverse Transforming the Image Perspective Back to its Original Position	25
4.10	Mapping the Line Marking Pixels onto the Original Image	25
4.11	Text File of x, y coordinates corresponding to the Line Marking	25
5.1	Image Instance from the Taxiway Dataset Used for Manual Labeling	28
5.2	Manually Drawn Contour Region Around the Line Marking	29
5.3	RGB to Grayscale Conversion of the Contour Region	29
5.4	Result After Applying Gaussian Blur to the Contour Region	29
5.5	Result After Applying Canny to the Contour Region	30
5.6	Shaded Region Indicating the TP Values (Intersection of Set A and Set B)	30
5.7	Shaded Region Indicating the FN values (Set Difference of B and A)	31

List of Tables

5.1	Illustration of the labeling process by ALINA for various scenarios on the taxiway	27
5.2	Results Obtained by ALINA	32

Abbreviations

ALINA	A utomated L ine I dentification and N otation A lgorithm
CIRCLEDAT	CIRC ular thresho L d pix E l D iscovery A nd T raversal
ROI	R egion O f I nterest
ATC	A ir T raffic C ontrol
UAV	U nmanned A erial V ehicle
AI	A rtificial I ntelligence
ML	M achine L earning
DL	D ee P L earning
CV	C omputer V ision
SW	S liding W indow
VP	V anishing P oint
PCA	P rincipal C omponen A lysis
HOG	H istogram of G radients
SVM	S upport V ector M achine
ACapsFPN	A ttentive CAPS ule F eature P yramid N etwork
IoU	I ntersection o ver U nion
GPS	G lobal P ositioning S ystem
HDR	H igh D ynamic R ange
NB	N aive B ayes
TPR	T rue P ositive R ate
FPR	F alse P ositive R ate
DBSCAN	D ensity - B ased S patial C lustering of A pplications with N oise
SIFT	S cale - I nvariant F eature T ransform
LBP	L ocal B inary P attern
EFM	E xtended F isher M odel

In Dedication to My Beloved Grandparents

Chapter 1

Introduction

1.1 Problem and Motivation

In recent times, the fully autonomous vehicles have become a center of attention of computer vision and robotics research. Mainly, camera-based lane detection is becoming an integral aspect towards perceiving the environment around vehicles, which is then employed in making decisions and guiding them on the appropriate path. In the aircraft industry, there have been many accidents which vary from crew distractions to communication with Air Traffic Control (ATC). This has increased the need for having an automated perception system, which can help the pilots in attaining more awareness of their surroundings while they are on the airport taxiway. As stated by the Plane Crash Info, 12% of all the aircraft accidents, between the years of 1959 and 2008, happened during the taxiing phase [1]. There could be many factors that may have caused this, such as poor weather conditions, low visibility and night operations, communication breakdown with the ATC, high traffic on the taxiway. In order to address this issue, some researchers have come up with solutions for assisting pilots during the taxiing phase, including: light-based guidance systems, which have been designed by the Federal Aviation Administration and Honeywell, computer vision based taxiway guidance techniques, Airport Moving Maps, and sensors, such as LIDAR and cameras mounted on an aircraft [12][27][5][22][2][30].

Very limited work has been done on the use of computer vision for line marking detection on an airport taxiway. One of the researchers proposed a solution, but their approach was constricted only towards using grayscale images of the taxiway and also the camera was placed on the nose wheel landing gear of an aircraft [29]. Aman Batra et al. [5] improved this work by proposing an approach for detecting the line markings using color images, and the camera was located on the vertical stabilizer of the aircraft. However, it was only tested using simulated video sequence of an airport environment in ideal daytime conditions, and the detection rate did not exceed 80%.

While looking into the enhancement of line marking detection for airport taxiways, it can be noticed that this problem is quite similar to detecting lane lines on a road. As a result, the various approaches that were used in the development of computer vision based systems for cars has been adopted for building ALINA. For example, an approach took advantage of the high brightness value of road markings compared to the rest of the road. Similarly, K.Y. Chiu and S.F. Lin exploited the color of line markings on the road with the help of a statistical procedure to differentiate the lane boundaries from the road. These prior works inspired this research to look into methods of extracting the line markings on an airport taxiway using color space transformation and statistical analysis.

1.2 Objectives

This research proposes ALINA for autonomously detecting the taxiway line markings. It has the ability to process large datasets consisting of taxiway images that are captured by a camera, mounted inside an aircraft. The main objective of this research is to provide solutions for:

- Eliminating manual intensive-labor from large-scale dataset labeling
- Enhancing safety by reducing the likelihood of runway incursions or collisions between aircraft
- Increasing efficiency by reducing the need for manual inspections by maintenance personnel or ground crew at the airport
- Creating reliable ML/DL models for taxiway line marking detection
- Improving the trajectory planning decisions

1.3 Contributions

Large datasets are being made available everyday, and this data can be used to drive consequential outcomes and make critical decisions using ML/DL methods. However, the biggest gap between acquiring the data and building an ML/DL model is the process of accurately labeling/annotating the data. Now, the process that is usually adopted for the labeling procedure is through manual intensive labor, but no matter how many people work on it, this is still a very time consuming process. In an attempt to overcome this problem and fully automate the labeling process, ALINA has been introduced as a technique that requires the dataset in jpg/png format as input and it produces annotations and text files of the locations of the line markings for each image in the dataset. While it is important to have the data labeled quickly, it is also critical to maintain a

high accuracy throughout the labeling process, as the results produced by this algorithm serve as a ground truth for substantiating future ML/DL models.

Initially, Sliding Window (SW) method was adopted in ALINA to test and gather information about how much time it actually took for finding the pixels of the line marking after the preprocessing stage. It resulted in a time complexity of $O(m * n)$, where m and n are the height and width of the image. In addition, this procedure required the image to be pre-processed in such a way that when it came to the step of utilizing the SW approach, only the pixels that related to line markings should be existing in the image, or else it would capture the noisy pixels as well. To significantly reduce computational complexity while simultaneously making ALINA resilient, the histogram analysis was performed on the images, followed by the introduction of a novel algorithm called CIRCLEDAT, to replace SW by identifying the pixels corresponding to only the line markings. The time complexity of CIRCLEDAT in its worst case scenario was recorded as $O(n)$.

1.4 Outline

The remainder of the thesis is organized as follows: In Section 2, prior research works based on contemporary computer vision techniques and system designs for acquiring lane markings on roadways and taxiways have been discussed in detail. Section 3 describes the research background and presents the tools and technologies that were utilized in building ALINA and creating the ground truth data for evaluation. Section 4 discusses the methodology behind ALINA, and Section 5 presents the experimental results obtained by it, including a comprehensive evaluation of its performance and comparison against existing state-of-the-art approaches. Finally, Section 6 provides the conclusion to the thesis, summarizing the key findings and contributions of the research and outlining future directions for further work.

Chapter 2

Literature Review

In this section, the previous research works has been discussed that have been done with regards to the development of line marking detection on roads and taxiways, the detection of horizon, and state-of-the art techniques that were utilized to extract meaningful information from an image.

A lane detection system was proposed which utilized a combination of probabilistic Hough transform and dynamic parallelogram ROI [3]. This approach could be applied only to a sequence of frames that were extracted from a video, where the ROI was defined manually on the initial frame of the video, and then probabilistic Hough transform was used inside the ROI to identify the line markings within it. Once the line marking was detected, the ROI took the shape of it, and consequently, the edge points outside of the ROI were suppressed in the subsequent frames. The drawback of this approach was that the probabilistic Hough transform could not capture endpoints of a curved line marking.

Tarui Chen et al. [9] proposed a machine learning-based solution for the detection and classification of road markings in a video-based intelligent transportation system. Their approach was comprised of two steps: the detection was carried out using binarized normed gradient method and the PCA network was employed for object classification. The proposed approach was trained and tested on 1,443 road images (60% training and 40% testing) and it could detect 9 classes of road markings and also proved to acquire an accuracy rate higher than 96.8%.

Principal Component Analysis- Histograms of Oriented Gradients (PCA-HOG) and Support Vector Machine (SVM) were integrated together for detecting and recognizing road markings in images [11]. The methodology initially included the steps of finding the vanishing point (VP), changing the perspective to a bird's eye view and applying segmentation using local median binarization. Subsequently, the HOG and PCA were combined to form a feature vector and then

classified using SVM. The accuracy and recall attained by this method were 94.77% and 82.91% respectively.

Haiyam Guan et al. [14] demonstrated an attentive capsule feature pyramid network (ACapsFPN) for extracting road markings from high spatial resolution UAV images. In order to obtain multi-level and multi-scale capsule features, the ACapsFPN incorporated capsule representations with attention mechanisms into the feature pyramid network. By combining the ternary feature attention modules and the multi-scale context feature descriptor, the network produced a class-specific feature representation. In comparison to other available options, the evaluation of the ACapsFPN demonstrated its effectiveness in extracting road markings from UAV images under various complicated scenarios. Precision, recall, IoU, and F1-score for the ACapsFPN were 0.7366, 0.7513, 0.5922, and 0.7439 respectively.

A framework for camera-based real-time lane and road surface marking detection and recognition was established [15]. The spatiotemporal incremental clustering, curve fitting, and Grassmann manifold learning were the foundations of it. A camera positioned on the front of a moving vehicle was used by the system to identify road surface markings, and detect lanes and traffic. Its performance was assessed using common datasets, and it was found to be on par with cutting-edge techniques.

Xinyu Jiao et al. [17] proposed a width-adaptive lane identification system that utilised the scan line method for ridge feature recognition, lane-voted VP for noise filtering, and multi-lane tracking Kalman filtering. It achieved an F1-Score of 93.4%. However, the algorithm was not very effective in differentiating road lanes from other objects with comparable edges in the image because it relied only on color and always required parallel lines to determine the VP. The researchers addressed that other items in the image were broader than the road lanes, but this may not always be the case in every scenario.

Jeffrey Kiske [18] discussed the development of an autonomous labeling system for identifying highway lane markings. The system created a tagged collection of lane markers using information from three sensors, including Velodyne lidar for depth, HDR video streams for color information, and high-precision GPS for location. The spots that weren't lane markings were deleted using thresholding after the sensor data was combined into a single map. The detected lanes were grouped together using hierarchical clustering, and a single point was added to each lane marking based on the median of each group. Although, neither a ground truth nor any metrics to assess the precision and accuracy of the suggested approach have been discussed in the paper.

A vision-based perception algorithm was proposed for lane detection in self-driving cars [19]. Initially, the algorithm included a minimalistic approach based on edge detection and polynomial regression for the detection of straight lane lines. Subsequently, an improved technique based on perspective transformations, histogram analysis and SW was developed for the detection of both

straight and curved lines, which proved to be more accurate and robust in various environmental conditions. However, the algorithm heavily depended on the SW mechanism, which had to process an entire frame every time, making the algorithm computationally very expensive. In addition, steep curves in the foreground could pose a challenge for the algorithm.

A texture feature-based approach for the lane detection used Gabor filters to estimate the texture orientation map at each pixel for locating the vanishing points in the images [20]. The pixels were then segregated by voting to generate a voting map, and the vanishing point was detected using peak identification in the voting map. Gabor filters were essentially used for extracting edge information and for getting rid of the noisy line segments that were found in the image, using the knowledge of VP.

A lane identification system employed various techniques such as pre-processing, thresholding, and perspective transformation to detect the lane boundary lines [24]. It achieved 96% accuracy in detecting lane lines on different roads under various illumination circumstances, and the false positive rate was just 0.640%.

A two step lane detection approach included global segmentation using color and texture data and a neural network, followed by local analysis which took advantage of the direction filter and length filter [28]. The color and texture data was retrieved prior to performing global segmentation with a neural network. The direction filter and length filter were subsequently employed in the local analysis approach for analyzing the road, as it was presumed that the information about the road had a definite direction and length. However, this approach was constricted to just straight roads or roads with little curvature

The detection and estimation of an aircraft's departure from the taxiway center-line was done using an onboard camera positioned on the aircraft's vertical stabilizer [5]. This approach initially applied homographic transform on the images, after which, the color and edge information were extracted. Once that was done, the sliding window (SW) method or clustering algorithm was applied to identify the taxiway line markings. The Density-Based Spatial Clustering of Applications with Noise (DBSCAN) clustering technique was used when the aircraft was turning in a direction whereas, the SW method was used to identify the taxiway center-line and any taxiway branching or junctions ahead of the aircraft. This methodology was tested on simulated video sequences and it obtained decision rates and false positives of 76% and 4%, respectively on the clustering technique, and 80% and 3% on the SW method, respectively.

Three line detection algorithms were compared, namely- the particle filter, the Hough transform, and the LaneNet neural network—for aircraft navigation on taxiways [22]. The comparison of these algorithms were carried out on both simulated and actual images, and it was based on factors like detection, precision and range. While the particle filter approach produced good results on simulated images, the research revealed that it was challenging to adapt it for actual

images or poor weather situations. The Hough transform approach had a strong color assumption and was more difficult to be apt in real images, whereas the LaneNet produced unsatisfactory results due to variances in line patterns.

The roads from aerial images were extracted using 2 steps: K-means clustering, for partitioning the road area from the rest of the objects in the image and dilation, a morphological technique, for filling any gaps between the road pixels [21]. The road was further classified based on the presumption that, in comparison to other objects in the image, it was the largest connected component. The recall, accuracy, and quality was assessed and calculated for 10 annotated images, and they were 93.46%, 79.28%, and 75.12% respectively. The approach would however, fall short in cities with complex environmental settings, necessitating further development.

The light-intensity gradients were employed to extract distinct light field and pinpoint the course of horizon in segmentation-based method for recognizing the horizon in images [6]. It grouped pixels with similar values into clusters and divided the intensity information into a smaller number of values. The technique had a 1.2% error rate when it was applied to 160 frames of UAV recordings. The methodology worked on the assumption that the image's light field is singular, which could provide misleading results.

SVM, J48, and Naive Bayes (NB) were used for distinguishing the sky and the ground, and subsequently determining horizon in the image [13]. Using an inequality condition, the features were designated, horizon line was projected, and pixels were labeled. Consequently, the optimal line for separating the sky from the earth was done with the help of the expectation maximization function. The accuracy of the classifiers was determined to be between 90% and 99%. This methodology however worked on the assumption that the horizon line was always a straight line.

The classification and clustering algorithms were integrated to detect the horizon line in images [25]. The texture properties in an image were initially calculated, and then a neural network was used to identify blocks as either sky or not. Then, depending upon the values of their intensities, pixels were grouped using pixel intensity clustering. If the relative size of the intersection was greater than a set threshold, the results of the neural network classification and clustering were integrated and merged into the sky region. The technique was tested using the NASA Basalt Hill dataset, and when the number of clusters were set to 10, a True Positive Rate (TPR) of 99.56% and a False Positive rate (FPR) of 2.82% were produced.

The various methods for extracting information from images, such as texture operators, color histograms, and shape descriptors, were listed in a survey along with their advantages and disadvantages [4]. It looked at the various ways in which these methods might be used to create a thorough description of the image content. The survey also provided a summary of the current state of the art techniques and highlighted research areas for further investigation.

Wilhem Burger and Mark Burgecite [8] discuss the Scale-Invariant Feature Transform (SIFT) technique, which is a local feature detection method that tries to discover visual features that can be robustly matched across many images and image sequences, as well as object recognition under diverse viewing situations. A rotation-invariant feature descriptor is connected to each candidate point in SIFT, which functions as a multi-scale corner detector with sub-pixel positioning accuracy. The feature descriptor acts as the "fingerprint" of the feature point by summarizing the gradient direction distribution in a geographic vicinity. The four basic steps of the SIFT algorithm are the identification of extrema in a Laplacian-of-Gaussian scale space, refinement of the key points, assignment of the orientation, and creation of the feature descriptor. The SIFT technique has been modified and used for acceleration. It is widely utilized in a variety of applications, including 3D reconstruction, image alignment, image matching in multiple images and image sequences, and object detection under various viewing situations.

A novel three-dimensional local binary patterns (3D-LBP) descriptor was proposed for encoding color and texture information in images [23]. This descriptor was merged with the HOG to produce a unique H-descriptor that encoded color, texture, shape, and local information. Next, the H-descriptor was extracted using PCA and the Extended Fisher Model (EFM). The object and scene image classification was performed using the closest neighbor criterion. The methodology was tested on UIUC Sports Event and MIT Scene datasets and the results were 86.2% and 90.8%, respectively.

Chapter 3

Research Background

A branch of artificial intelligence (AI) called computer vision makes it possible for machines to understand and evaluate visual data from the outside environment. It has the ability to tackle a wide range of problems, including object identification, image categorization, face detection, and tracking, among others, with the use of sophisticated algorithms and deep learning models [32]. These skills have numerous uses in robotics, self-driving automobiles, unmanned aerial vehicles (UAV's), medical imaging, and surveillance.

Computer vision has come a long way since its inception as a mathematical approach to interpreting and analyzing visual data. It has developed from its earliest iterations of straightforward image processing algorithms into a sophisticated area of study that now includes a variety of methods and uses [10]. Despite the current focus on data-driven strategies and deep learning, the core ideas of computer vision continue to have their roots in mathematics and computer science [31].

As in this research, using the computer vision techniques has aided towards identifying and extracting line markings from the images of an airport taxiway, taken from inside an aircraft. Once the images in the taxiway dataset have been labeled, they can be further utilized to build machine learning or deep learning models to guide the aircraft safely and precisely to and from its starting point or destination.

In addition, scalability is one of the key benefits of employing computer vision for building an automated data labeling system. It has the ability to classify massive datasets fast and accurately because it can analyze enormous volumes of data, whereas humans would need a lot of time and intensive manual labor to do the same work. Moreover, because the algorithms are created to recognize and extract particular information from an image, employing computer vision provides a high level of accuracy and precision.

As technology is growing exponentially, the development of autonomous systems is becoming increasingly vital. Self-driving cars and UAV's are just a few examples of autonomous systems that rely on accurate and efficient data labeling to navigate their environment. While ML algorithms have typically been employed for data labeling, the applications of computer vision have made it a more viable option.

Unlike ML, computer vision does not require pre-existing labeled data, instead, it uses mathematical approaches to identify and extract features. By exploiting the potential of computer vision, more efficient and accurate autonomous systems can be designed, opening the path for a future when machines can safely and effectively traverse the world around people.

3.1 OpenCV

The open-source software library OpenCV is crucial for computer vision and machine learning applications. It includes a comprehensive range of tools for real-time image and video processing, including object recognition, feature detection, image filtering, and tracking [7]. Its vast variety of functionalities has been critical in the development of an automated data labeling system for autonomous aircraft perception. ALINA has the ability to process and label large volumes of image data taken by cameras located inside the aircraft.

A number of OpenCV functions have been used while building ALINA. `Imread()` and `imshow()`, for example, have been helpful for visualizing and analyzing images, whereas `inRange()` was essential for filtering and segmenting the images based on color ranges. Additionally, geometric transformations—important phases in the data labeling process—required the `polylines()` and `warpPerspective()` algorithms. The `normalize()` method was essential for preparing the images and for separating the item of interest in order to extract important characteristics. Each function that has been used from the OpenCV library are briefly explained in the parts that follow.

3.1.1 Loading and Displaying Images

3.1.1.1 `imread()`

The `imread()` function is used to read an image file. The method returns a NumPy array that represents the image pixels and accepts the path to the image file as an input. The first of the function's two inputs is the location of the image file, and the second is an optional flag that specifies how the image should be read. The parameter "flag" has three possible values:

- `cv2.IMREAD_COLOR` : loads the image in the RGB format.

- `cv2.IMREAD_GRAYSCALE` : loads the image in grayscale format.
- `cv2.IMREAD_UNCHANGED` : loads the image as it is (with alpha channel).

As processing images in Python is essential for many computer vision applications, the `imread()` method is important. This feature allows to manipulate images in a variety of ways, including scaling, cropping, filtering, and more.

3.1.1.2 `imshow()`

The `imshow()` function is used to show an image in a window. The function requires two arguments: the window's title as the first parameter and the image to be shown as the second. The processed image can be seen with this function, which is important for debugging computer vision programs. To display the image at different stages of processing, the `imshow()` method is frequently used in conjunction with other OpenCV functions like `imread()`, `cvtColor()`, `.threshold()`, etc. The third optional argument for the `.imshow()` method is the delay time in milliseconds. This option indicates the amount of time the window should be left open before closing on its own.

3.1.1.3 `imwrite()`

The `imwrite()` function is used to save an image to a file. The first parameter of the `imwrite()` method is the path to the file where the image has to be saved, and the second parameter is the image itself. The function allows in storing the processed image to a file, which is important for many computer vision applications. The third parameter of the `imwrite()` function is optional and defines the level of compression for the stored image. Only when the image is being saved in a compressed format like JPEG is this option applicable. The compression quality option has a value between 0 and 100, with 0 denoting the most compression (lowest quality) and 100 denoting the lowest compression (highest quality).

3.1.1.4 `waitKey()`

The `waitKey()` method is used to wait for a keyboard event to happen for a predetermined period of time. The milliseconds to wait in is the only input for the function. The method waits endlessly until a key is touched if the argument value is 0. The function allows to display an image or video and wait for user feedback before continuing with additional processing or shutting the window. It waits for a keyboard event to happen for the length of time that is given. If a key is pressed while the function is waiting, the method returns the key's ASCII value. The method returns -1 if no key is pushed during the waiting period. The function is frequently used in video processing and interactive applications.

3.1.1.5 `destroyAllWindows()`

The `destroyAllWindows()` method is used to delete all windows that were produced by OpenCV. The function accepts no arguments. The function can shut all OpenCV-created windows simultaneously without having to do it one at a time. When working with several windows or closing a window that wasn't closed correctly, this is useful. It deletes all windows from the screen and releases all resources related to them. It is frequently used at the end of the program or script to make sure that all windows are closed before a program or script ends.

3.1.2 Image Processing

3.1.2.1 `cvtColor()`

The `cvtColor()` function is used to change the color space of an image. The image that has to be converted is the first argument, and the second parameter is the conversion code, which indicates the sort of conversion that needs to be done. The function plays the role of manipulating the image's colors in a number of ways. There are several possible choices for the conversion code argument, including `cv2.COLOR_BGR2GRAY`, `cv2.COLOR_BGR2HSV`, `cv2.COLOR_BGR2LAB`, and others. Each code denotes a certain color space conversion process.

3.1.2.2 `polylines()`

The `polylines()` function is used to draw a polygonal curve on an image. The function requires four inputs: the image on which the curve has to be created, a list of points defining the polygon's vertices, a Boolean value indicating whether the curve is a closed polygon or an open curve, and a tuple specifying the curve's color. The function makes it possible to draw intricate forms on the image. In order to carry out different image processing tasks, the `polylines()` method is frequently used in conjunction with other OpenCV functions like `imread()`, `imshow()`, `cvtColor()`, etc.

3.1.2.3 `inRange()`

The `inRange()` method is used to extract a binary mask from an input image based on a predetermined range of colors. The function requires three parameters: the input image, the lower and upper bounds of the color range. The function helps in extracting particular regions of interest from the image and segment it depending on color. It checks each pixel of the input image with the provided color range, and based on whether the pixel color is within the range or not, assigns a binary value of 1 or 0 to each pixel in the output image. The output binary mask

can be employed for different image processing tasks including object recognition, tracking, and segmentation.

3.1.2.4 bitwise_and()

The `bitwise_and()` method performs a bitwise logical AND operation between two input images or binary masks. The first parameter of the function is the first input binary mask or image, and the second parameter is the second input binary mask or image. The function helps us to combine binary masks or images depending on the pixel values they correspond to. By comparing the pixel values of the two input images or masks, the `bitwise_and()` method determines which pixel value should be applied to the output image or mask. By executing a bitwise logical AND operation across the matching pixels of the input images or masks, the output image or mask is produced. The function is widely used for tasks such as image masking and segmentation.

3.1.2.5 split()

The `split()` method is used to separate a multi-channel image into its component color channels. The input image is the only parameter required by the function. The function gives a distinct control over each color channel in an image. The multi-channel input image is divided into its constituent color channels using the `split()` method, and these channels are then returned as independent single-channel images. One of the color channels, such as Red, Green, or Blue, is represented by each of the images that are returned. Tasks involving the processing and analysis of color-based images frequently employ this function.

3.1.2.6 normalize()

The `normalize()` method is used to normalize an input image or matrix. The function requires five parameters: the input image or matrix, the output image or matrix, the minimum and maximum values of the normalization range, and the kind of normalization. The function enables in scaling and shifting an image's or matrix's intensity values to a certain range. With this method, images' contrast and visibility may be improved. It can also be used to get matrices ready for other processing, including machine learning techniques. L1 and L2 normalization are two of the many forms of normalizing that the function offers.

3.1.3 Geometric Transformations

3.1.3.1 `getPerspectiveTransform()`

The `getPerspectiveTransform()` method is used to determine the perspective transformation matrix between two planes in an image. The function accepts two inputs: a list of four points in the source plane as the first argument, and a list of four points in the destination plane as the second. The function enables in fixing perspective distortion in the image brought on by different camera perspectives and angles. The input points are used by the `getPerspectiveTransform()` method to determine the transformation matrix that may be used by the `warpPerspective()` function to correct the image. The 3x3 matrix that represents the transformation between the two planes is the result of the `getPerspectiveTransform()` method.

3.1.3.2 `warpPerspective()`

The `warpPerspective()` method is used to give an image a perspective modification. The function accepts three parameters: the source image, the transformation matrix acquired by `getPerspectiveTransform()`, and the output image size. The source image is the first parameter. The input image is transformed using the transformation matrix by the `warpPerspective()` method to produce the corrected image. The function also allows extra arguments like the interpolation method to use for resampling the image, and the output image size may be supplied to crop or enlarge the image.

3.2 NumPy

NumPy is a python package for numerical and scientific computation. It has extensive array processing capabilities, which are necessary for scientific computing workloads like linear algebra, Fourier analysis, and statistics. It can handle and manipulate massive numerical datasets. The NumPy arrays aid in storing and manipulating the pixel values of images. One of its key features is the ability to execute element-wise operations on arrays, which enables effective calculations on enormous datasets. In addition, it has a large selection of mathematical operations that may be used on arrays, such as trigonometric, exponential, logarithmic, and statistical operations.

NumPy has been used in this research to carry out a number of operations on image data, such as filtering, masking, identifying the indices of required pixels in the images, etc. One of the most frequently used NumPy methods, `reshape()`, allows for effective array reshaping. Certain characteristics can be extracted from an image by choosing particular components from an array using NumPy's indexing and slicing capability.

In general, this package is a vital tool for scientific computing and data manipulation because it offers robust array processing capability that makes it possible to compute effectively on big datasets. It has been the best option for building ALINA because of its adaptability, simplicity, and large variety of mathematical operations. It is famously referred to as `np` while it is imported during programming. The following functions from this package have been used in this research:

- `np.array()`: It converts a sequence, such as a list or tuple, into a NumPy array object. It can also take optional arguments like `dtype`, `copy`, etc. to specify the data type of the array and other parameters.
- `np.int32()`, `np.float32()`, `np.uint8` (`dtype`): When building a NumPy array or casting an existing array, several data types might be supplied. A 32-bit integer data type, a 32-bit floating-point data type, and an 8-bit unsigned integer data type are all created using the `np.int32()`, `np.float32()`, and `np.uint8()` functions, respectively.
- `np.ones()`: It produces a NumPy array made up entirely of ones. The shape of the array can be specified with the optional `shape` argument.
- `np.where()`: It provides the indices where a given condition is true. It takes a condition as its first input, and it also accepts optional parameters like `x` and `y` that specify what should be returned in the event that the condition evaluates to true or false.
- `np.stack()`: It stacks arrays along a new axis. A list of arrays is the first input, and an optional argument specifying the axis along which to stack the arrays is the second.
- `np.sum()`: It returns the sum of all elements in a NumPy array. It can take optional arguments like `axis` to specify the axis along which to sum the elements.
- `np.max()`: It returns a NumPy array's largest element. It can take optional arguments like `axis` to specify the axis along which to find the maximum.
- `np.argmax()`: It returns the index of the largest NumPy array element. It can take optional arguments like `axis` to specify the axis along which to find the maximum index.

3.3 Matplotlib

Matplotlib is a data visualization library for creating static, animated, and interactive visualizations in Python. It provides a variety of functions to create different types of plots, graphs, and charts. In this research, the `plt.plot()` function has been used while calculating the histogram for projecting the required data from the image, and `plt.xlabel()` and `plt.ylabel()` to label the `x` and `y` axes of the plot, respectively. The `plt.show()` is used to display the plot on the screen. These functions help us to visualize and analyze the results of the image processing algorithm.

In summary, Matplotlib is a powerful data visualization library that provides a variety of functions to create different types of plots, graphs, and charts. The `plt.plot()`, `plt.xlabel()`, `plt.ylabel()`, and `plt.show()` functions used while building ALINA are just a few examples of the many functions provided by Matplotlib to create customized visualizations.

3.4 Canny Edge Detection Algorithm

The Canny edge detection algorithm is a widely-used method for detecting edges in an image. In this research, it was utilized to create the ground truth against which ALINA's performance was evaluated. It produced the precise location edges of the taxiways line markings.

When using Canny edge detection, there are typically four steps involved. First, the image is denoised to reduce any noise using a Gaussian smoothing filter using the equation 3.1.

$$G(x, y) = \frac{1}{2\pi\sigma^2} * e^{-\frac{x^2+y^2}{2\sigma^2}} \quad (3.1)$$

Second, the gradient amplitude and direction are calculated as shown in equations 3.2 and 3.3, with the gradient direction typically taken at four angles - 0, 45, 90, and 135 degrees.

$$\sqrt{G_x^2 + G_y^2} \quad (3.2)$$

$$G = \arctan \frac{G_y}{G_x} \quad (3.3)$$

Third, non-maximum suppression is applied to eliminate non-edge pixels, leaving only a few fine lines. Finally, the hysteresis threshold is selected, which requires two thresholds to retain or exclude pixels and select the edge.

If the amplitude of the pixel position is higher than the high threshold, the pixel is reserved as an edge pixel. Conversely, if the amplitude of the pixel position is less than the high threshold, the pixel is excluded. If the amplitude of the pixel position is between the two thresholds, the pixel is reserved only when connected to a pixel higher than the high threshold. Overall, Canny edge detection involves several steps to accurately detect edges in an image, including denoising, gradient calculation, non-maximum suppression, and hysteresis thresholding.

Chapter 4

Methodology

This section aims to provide a comprehensive understanding of ALINA. The flowchart in Fig. 4.1 represents the various stages involved in the process. Using this flowchart and an image instance from the acquired dataset, each step of ALINA will be described in detail.

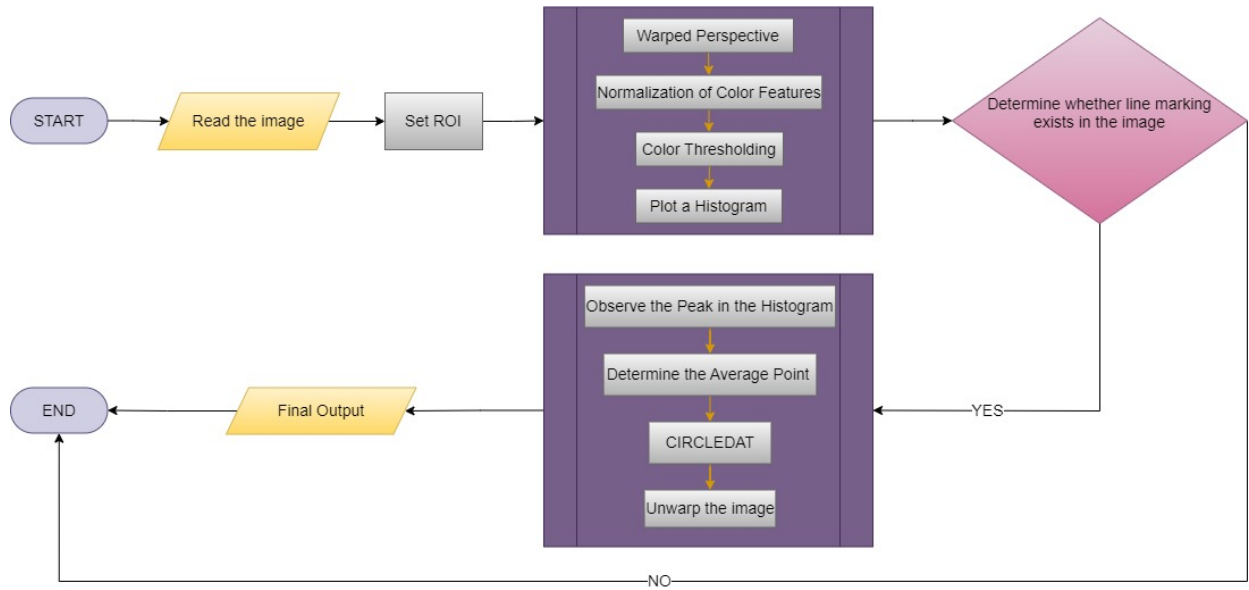


FIGURE 4.1: Flowchart of ALINA

4.1 Image Representation

ALINA begins by storing each x, y coordinate and its RGB channel values from the image into a multi-dimensional array as shown in equation 4.1.

$$A[i, j, k] = I[i, j, k] \quad (4.1)$$

where, i , j , and k denote the row, column, and channel indices of the image, respectively. This formula copies the pixel values of the image at location (i, j) in the k^{th} color channel into the corresponding location in the array. ALINA can now represent the image as a multi-dimensional array, allowing it to perform numerical operations and manipulations on the image data. Fig. 4.2 illustrates the representation of an image instance from the dataset, which will undergo the pre-processing in the subsequent steps.



FIGURE 4.2: Image Instance from the Taxiway Dataset

4.2 Interactive ROI Definition for Image Analysis

The user provides four points to ALINA for creating an inverse trapezoidal ROI in the vicinity of where the camera expects to see the taxiway's line markings. Fig. 4.3 shows where the ROI for Fig. 4.2 has been defined. ALINA treats the ROI as a constraint to limit the search area for taxiway's line markings, resulting in reduced computational complexity and improved detection accuracy [16].



FIGURE 4.3: ROI highlighted in red on the image

4.3 Advanced Perspective Transformation of the ROI

The trapezoidal ROI is warped into a bird's eye view as shown in Fig. 4.4. This transformation enables a more comprehensive and consistent view of the taxiway and its features, allowing for improved detection of line markings irrespective of their orientation or curvature [24]. The bird's eye view refers to a top-down perspective of the region, as seen from a high vantage point. This view appears as a rectangular shape, which facilitates further processing of the image data.

The transformation is achieved using a perspective transformation matrix, denoted by M , which is computed using the source and destination points. The source points are four corners of the trapezoidal ROI, and the destination points are four corners of a rectangular shape, which represents the bird's-eye view of the ROI. Equation 4.2 represents the calculation of the perspective transform matrix.:

$$M * [x, y, 1]^T = [x', y', 1]^T \quad (4.2)$$

where $[x, y, 1]^T$ is a homogeneous coordinate vector representing a point in the source image, $[x', y', 1]^T$ is a homogeneous coordinate vector representing the corresponding point in the destination image, and M is the 3×3 perspective transformation matrix.

The calculated 3×3 matrix M is then used to transform the image as shown in equation 4.3:

$$t(x, y) = s \left(\frac{M_{11}x + M_{12}y + M_{13}}{M_{31}x + M_{32}y + M_{33}}, \frac{M_{21}x + M_{22}y + M_{23}}{M_{31}x + M_{32}y + M_{33}} \right)$$

where $t(x, y)$ are the coordinates of pixels in the transformed image and $s(x, y)$ are the coordinates of pixels in the source image.



FIGURE 4.4: Bird's Eye View of the ROI

4.4 Color Feature Normalization for Enhanced Image Processing

To accurately distinguish line markings from the rest of the image, normalizing color characteristics is a crucial step in ALINA. The hue, saturation, and value (HSV) color channel is used to normalize the influence of various lighting conditions and color values. Hue represents the actual color of an object in the image, based on its wavelength. Saturation measures the intensity or purity of the color, where a fully saturated color appears more vivid and a desaturated color appears more washed out. Value (or brightness) measures the lightness or darkness of the color. Together, these three components provide a more complete representation of color information in an image [26].

The normalization phase involves the following steps:

1. Transforming the RGB image to HSV color space
2. Dividing the image into H, S, and V components
3. Normalizing each component individually
4. Stacking the normalized components back together

This process standardizes the color feature values across the image as shown in Fig. 4.5, ensuring that each feature contributes equally to the final representation and, as a result, facilitates the employment of a color thresholding approach to identify line markings.

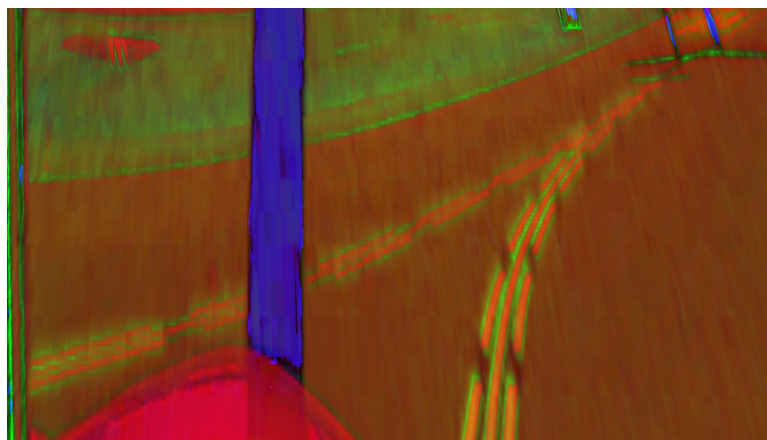


FIGURE 4.5: Color Feature Normalization using HSV components

4.5 Color Thresholding for Image Segmentation

Color thresholding is a technique used to segment an image based on its color information. In ALINA, it is used to segment the line markings from the rest of the image content. The hue

component of the image was utilized to determine the yellow threshold, and not the saturation and value components. This is because, it was found that the hue component provided a more reliable indication of the line markings, capturing the wavelength of the color. On the other hand, using the saturation and value components resulted in other significant objects or areas in the image to fall within the same threshold range as the line markings. The lower and upper limits of the yellow hue values established empirically were (0, 70, 170) and (255, 255, 255), respectively. The pixels that fell in this range were marked as white pixels (255), while the others were marked as black pixels (0). However, even after using only the hue component to differentiate the line markings, there were still some areas in the image that fell into that same range, as shown in Fig. 4.6. ALINA tackles this challenge by further processing the image in subsequent steps.



FIGURE 4.6: Color Thresholding based on Hue Component

4.6 Analyzing Image Data Distribution with Histogram Plotting

The color thresholded image's histogram is plotted to determine the area of the most densely packed white pixels along the y-axis, which reflects the location of the line marking in the image. In order to attain this, initially the image is converted into a binary representation, where white pixels are represented as 1 and black pixels as 0. Then the sum of each column in the binary image is obtained, to get a projection profile of white pixels along the vertical axis. Finally, the projection profile is plotted as a histogram, with the x-axis representing the horizontal coordinates of each column in the image, and the y-axis representing the number of white pixels in each column. This projection illustrates the existence and location of the line marking in the image as shown in Fig. 4.7.

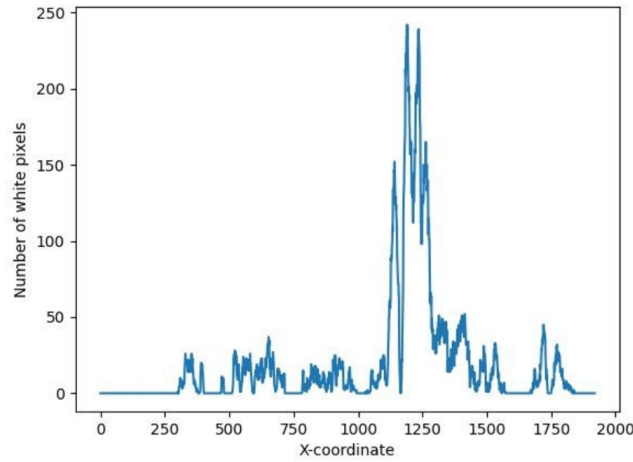


FIGURE 4.7: Histogram of white pixels across the x-axis in the image

4.7 Identifying Line Markings and Avoiding False Positives

After plotting the histogram, the peak value and index are determined. A conditional statement is applied to the peak value to determine whether there is a line marking in the image. A threshold is set on the peak value to determine the existence of a line marking, and if it is below that threshold, ALINA will output that there is no line marking and generate an empty text file. If the number of white pixels is above the threshold, further processing is performed on the image. Overall, this stage assists majorly in classifying the existence of a line marking, and if there is one, it proceeds to extract a mean x, y coordinate on that line marking for using it as an initial point in the next step of ALINA.

4.8 CIRCLEDAT: CIRCular threshoLd pixEl Discovery and Traversal Algorithm

The aim of this step is to isolate the pixels that correspond to the line marking and remove all other pixels from the image. This process ensures that only the relevant information is retained. Previous researchers have utilized the SW approach to detect pixels corresponding to line markings [5][19]. However, their approach was computationally very expensive, and to address this issue, CIRCLEDAT algorithm has been proposed, which is embedded in ALINA.

CIRCLEDAT's objective is to just extract the pixels corresponding to line marking from the image. This approach entails locating and moving through the pixels inside a circle-shaped threshold that surrounds a specified starting point. A visited array is used by the algorithm to maintain track of which pixels have previously been visited and a stack data structure to keep track of the pixels that need to be processed. If a pixel is recognized as a component of the line

marking, it is added to a list of marked pixels of the line marking. The algorithm then scans the pixels immediately surrounding the current pixel, inside the circular threshold, and adds those that have not yet been visited to the stack for processing.



FIGURE 4.8: Result of Applying CIRCLEDAT on the Thresholded Image

Algorithm 1 Circular Threshold Pixel Discovery and Traversal

```

1: function CIRCLEDAT(image, x, y, threshold, visited, line_marking_pixels)
2:   stack  $\leftarrow [(x, y)]$ 
3:   visited[y][x]  $\leftarrow 1$ 
4:   circular_range  $\leftarrow \text{list}(\text{product}(\text{range}(-\text{threshold}, \text{threshold} + 1), \text{repeat} \leftarrow 2))$ 
5:   circular_range.remove((0, 0))
6:   while stack is not empty do
7:     x, y  $\leftarrow \text{stack.pop}()$ 
8:     if image[y][x] = 255 then
9:       add (x, y) to line_marking_pixels
10:      for i, j  $\leftarrow \text{circular\_range}$  do
11:        x_new  $\leftarrow x + i$ 
12:        y_new  $\leftarrow y + j$ 
13:        if visited[y_new][x_new] = 1 then
14:          continue
15:        end if
16:        if x_new, y_new < 0 or x_new, y_new  $\geq \text{image.width}, \text{image.height}$  then
17:          continue
18:        end if
19:        add (x_new, y_new) to stack
20:        visited[y_new][x_new]  $\leftarrow 1$ 
21:      end for
22:    end if
23:  end while
24:  return line_marking_pixels
25: end function

```

The input to CIRCLEDAT is an image, initial *x* and *y* coordinates, a threshold value, a visited matrix, and an array for storing the white pixels that correspond to the line marking. CIRCLEDAT begins by adding the starting point to a stack and marking it as visited. It then

enters a loop, where it pops the top element from the stack and checks if the corresponding pixel in the image is white. If the pixel is white, the algorithm adds the pixel coordinates to an array that is used to store the line marking pixels. Then, the surrounding pixels within a circular range of that coordinate are examined. The circular range is determined by the threshold parameter. CIRCLEDAT adds any unvisited white pixels within the circular range to the stack, making sure that only the white pixels in the image that are within the threshold distance are taken into account for the line marking. Now, after a white pixel is popped from the stack and added to the line marking pixel array, CIRCLEDAT proceeds to traverse the surrounding pixels within the circular range until all the white pixels within the threshold distance have been added to the stack. Once this process is complete, the algorithm pops another white pixel from the stack and repeats the process of examining the surrounding pixels. This continues until there are no more white pixels in the surrounding and the stack is empty. The final output is an array that contains all the white pixels that correspond to the line marking. The visited matrix is used to keep track of which pixels have already been visited to avoid re-examining them. If a pixel has already been visited, it is skipped. If a pixel is outside the image dimensions, it is also skipped. CIRCLEDAT outputs the line marking pixel array, which only contains the white pixels that match the line marking. It uses a stack data structure to implement the recursive process of examining pixels within the circular range, ensuring that it can capture all white pixels that fall in the vicinity of the circular threshold.

4.9 Image Unwarping and Marking the Line Markings Pixels

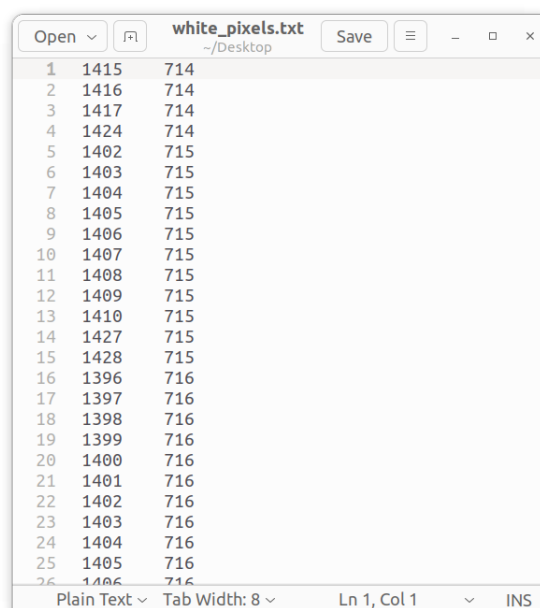
The final stage of ALINA involves an inverse perspective transform that returns the warped image to its original view, as shown in Fig. 4.9. This process involves computing the inverse perspective transform matrix using the previously determined destination and ROI points. After obtaining the unwarped image, white pixels are located and mapped onto the original image using the color red to clearly identify the line markings, as shown in Fig. 4.10. This representation provides a precise and clear depiction of the line markings in the original image. In addition, a text file is generated containing the coordinates of all pixels corresponding to the line markings, as shown in Fig. 4.11. This step provides a convenient output format for further analysis and building ML/DL models in the future.



FIGURE 4.9: Inverse Transforming the Image Perspective Back to its Original Position



FIGURE 4.10: Mapping the Line Marking Pixels onto the Original Image



	x	y
1	1415	714
2	1416	714
3	1417	714
4	1424	714
5	1402	715
6	1403	715
7	1404	715
8	1405	715
9	1406	715
10	1407	715
11	1408	715
12	1409	715
13	1410	715
14	1427	715
15	1428	715
16	1396	716
17	1397	716
18	1398	716
19	1399	716
20	1400	716
21	1401	716
22	1402	716
23	1403	716
24	1404	716
25	1405	716
26	1406	716

FIGURE 4.11: Text File of x, y coordinates corresponding to the Line Marking

Chapter 5

Experimental Results

5.1 ALINA Performance: Specifications and Scenarios

In this research, a total of 60,249 images were labeled using ALINA. The labeling process was carried out using a Linux system, which had the x86_64 architecture and an Intel Core i7-9700K CPU running at 3.60GHz. The system had a total of 15GB of RAM and was running Ubuntu 22.04.1 LTS. ALINA was developed using the PyCharm IDE, with Python version 3.8.15 as the programming language. Several libraries, including OpenCV, NumPy, Matplotlib, os, statistics, and time were used by ALINA during the labeling process. In addition, the CPU of the Linux system had a minimum clock speed of 0.8GHz and a maximum clock speed of 4.9GHz. These specifications emphasize the high computing power of the system, enabling efficient and accurate labeling of the images with ALINA.

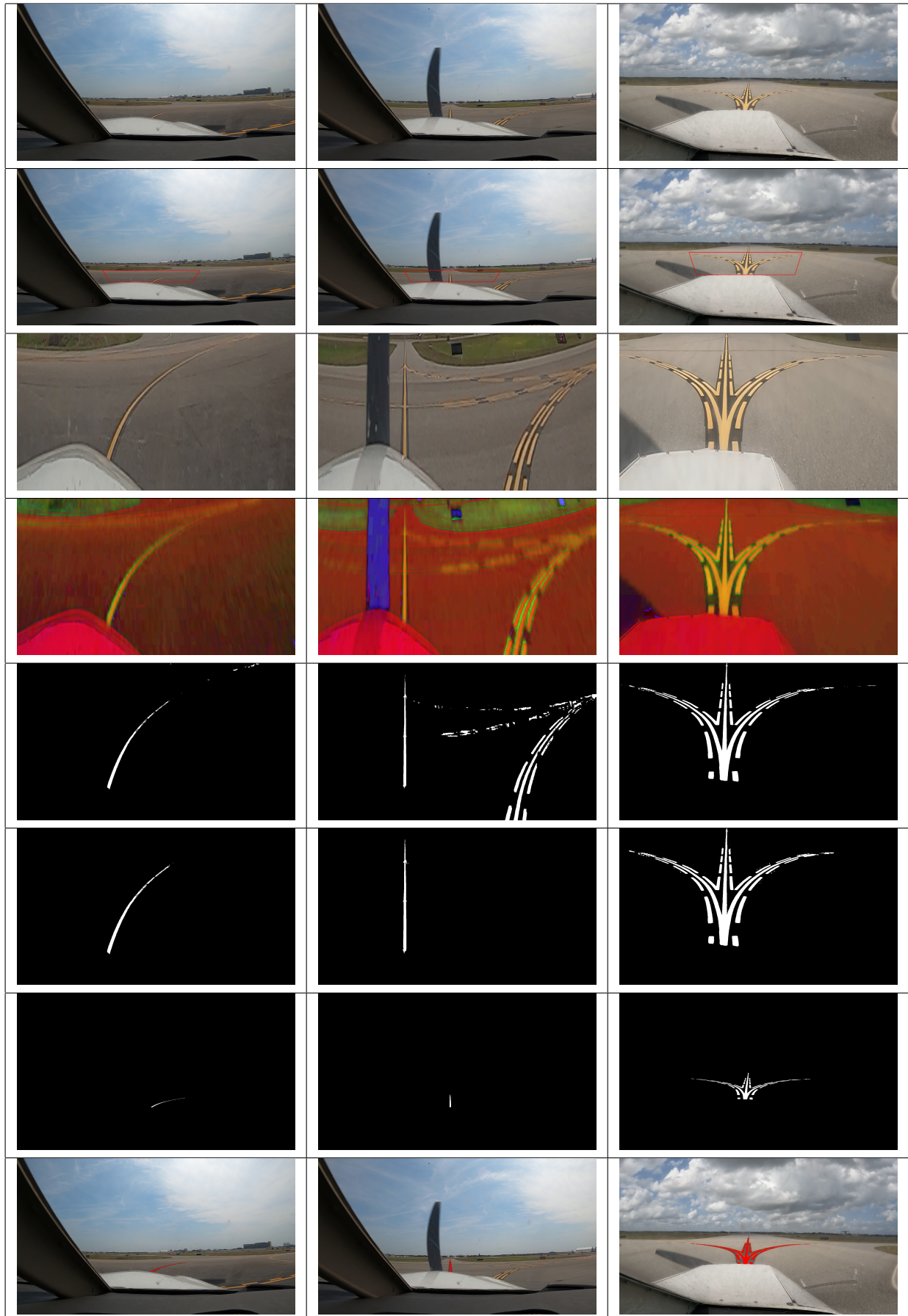
The total of 60,249 images were gathered from different videos which had different camera perspectives. The distribution was as follows: 42,419, 5,652, and 12,178. To demonstrate the effectiveness of ALINA, Table 5.1 showcases the labeling process of an image from each of the data distributions, highlighting ALINA's consistent performance regardless of the camera angle.

The first image scenario in the left column is the region on the airport taxiway when the aircraft was holding short, waiting for instructions from ATC to proceed further. While it was at this point, ALINA detected the line marking that was ahead of the aircraft to show in which direction it needed to go, when the pilot would get the signal from the ATC.

The second image scenario in the center column shows that the aircraft is following the taxiway lines which are on the straight path ahead, irrespective of the other line markings on the taxiway.

The third image scenario in the right column shows that there are three taxiway line markings ahead of the aircraft, and it has three choices, i.e. it can either go left, straight or right.

TABLE 5.1: Illustration of the labeling process by ALINA for various scenarios on the taxiway



5.2 Ground Truth Generation

To quantify ALINA's performance, it was necessary to establish a ground truth. The edges of the line marking alone could provide precise information about the location of the line marking within the image. As such, the inside x, y coordinates of the line marking were deemed unnecessary while creating the ground truth. To achieve this, the Canny edge detection algorithm was utilized, with 120 images randomly selected from the taxiway dataset based on the variability and scenarios captured by the camera. The first step involved using the cursor to create a contour around the line markings, after which the area was converted to grayscale and subjected to Gaussian blur. The Canny algorithm was then applied with a lower and upper threshold of 30 and 150 respectively, which was empirically achieved, successfully extracting the white-colored edges of the line markings while rendering the remaining pixels black. The x and y coordinates of the white pixels were then extracted and saved in a text file for that image. This process was carried out for 120 images, generating 120 text files, each providing the ground truth for its respective image. To ensure the correctness of the ground truth, the contour needed to be carefully drawn to capture only the edges of the line markings and not any other edges in the image. The manual process of outlining the contour region around the taxiway line markings and the consequent steps are illustrated in Fig. 5.1, 5.2, 5.3, 5.4, and 5.5.



FIGURE 5.1: Image Instance from the Taxiway Dataset Used for Manual Labeling



FIGURE 5.2: Manually Drawn Contour Region Around the Line Marking



FIGURE 5.3: RGB to Grayscale Conversion of the Contour Region



FIGURE 5.4: Result After Applying Gaussian Blur to the Contour Region



FIGURE 5.5: Result After Applying Canny to the Contour Region

5.3 Detection Performance: ALINA vs Ground Truth

To evaluate the performance of ALINA in detecting line markings accurately, it was necessary to compare its output with the ground truth, which was generated by Canny edge detection algorithm. The ground truth provided a reference point for the correct coordinates of the line markings. By using the x, y coordinates of both the ground truth and ALINA's output, the true positive (TP) and false negative (FN) values were determined. True positives represented the correctly detected line markings by ALINA, while false negatives indicated the line markings that were present in the ground truth but not detected by ALINA.

Let set A and set B represent the x, y coordinates of the line markings generated by ALINA and the ground truth, respectively. Then the intersection of set A and set B produces the TP values, and the set difference of B and A provides the FN values, as shown in Fig. 5.6 and 5.7, respectively. This comparison allows for an accurate assessment of ALINA's performance in detecting line markings on a given image.

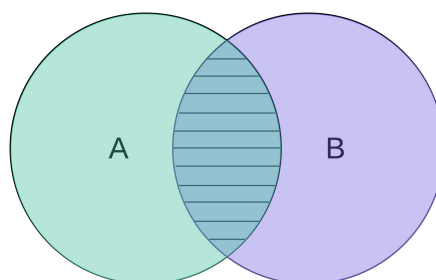


FIGURE 5.6: Shaded Region Indicating the TP Values (Intersection of Set A and Set B)

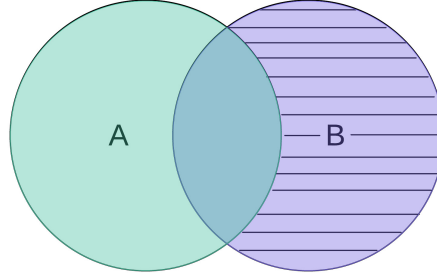


FIGURE 5.7: Shaded Region Indicating the FN values (Set Difference of B and A)

The recall or detection rate is a key metric for assessing the effectiveness of object detection algorithms. It measures an algorithm's ability to correctly identify all relevant objects in an image and is particularly crucial in line marking detection on an airport taxiway. A missed line marking can potentially lead to safety hazards, making it essential to identify all markings accurately.

The detection rate is calculated as the ratio of TP values to the sum of TP and FN values, as shown in equation 5.1.

$$\text{Detection Rate} = \frac{TP}{TP + FN} \quad (5.1)$$

It considers both the correctly detected line marking coordinates (TP's) and the missed coordinates of the line marking (FN's) to provide a measure of an algorithm's completeness in detection performance. A high detection rate indicates that the algorithm can accurately detect most of the coordinates of the line markings in the image, while a low detection rate suggests that it is missing many of the line marking coordinates. The detection rate obtained by ALINA after testing it with the 120 images of the ground truth was recorded as 0.9844, therefore, demonstrating its high reliability and effectiveness in detecting line markings on the airport taxiway.

Precision, on the other hand, is not the most suitable metric for line marking detection on an airport taxiway, because it is essentially a measure of the algorithm's prediction rate. Precision measures the proportion of detected objects that are correctly identified but does not take into account the objects missed by the algorithm (FN values). In line marking detection, it is essential to accurately detect all markings already present in the image, making detection rate the more appropriate metric.

The prediction rate (precision) is calculated as the ratio of TP values to the sum of TP and False Positive (FP) values, as shown in 5.2.

$$Prediction\ Rate = \frac{TP}{TP + FP} \quad (5.2)$$

After testing ALINA with 120 ground truth images, its prediction rate was measured to be 0.9229. ALINA's performance with regard to the prediction rate was slightly lower than the detection rate because the ground truth only provided information about the edges of the line marking, while ALINA not only generated the edges of the line marking but also the x and y coordinates inside it. As FP values were taken into account for the calculation of the prediction rate, the correct coordinates inside the line marking detected by ALINA were being treated as FP values.

Using the detection rate (recall) and prediction rate (precision), F1-Score was consequently calculated using the formula shown in equation 5.3, and it was recorded as 0.9527.

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (5.3)$$

The Table 5.2 showcases all the results obtained by ALINA. It has outperformed Aman Batra and Jason Gauci's proposed method for detecting line marking pixels in images. Their method achieved detection rates of 80% and 76% using the SW and DBSCAN methods respectively, where they applied the DBSCAN clustering technique to binary maps when the aircraft was turning, and used the SW method to detect the taxiway centerline when it was not. In contrast, ALINA uses the CIRCLEDAT algorithm to address all possible scenarios, including detecting straight and curved line markings. Also, ALINA has achieved a significantly higher detection rate of 0.9844 on the real-world taxiway images, whereas their results were based on simulated images under ideal daytime conditions.

TABLE 5.2: Results Obtained by ALINA

Metrics	Results
Detection Rate	0.9844
Prediction Rate	0.9229
F1-Score	0.9527

Chapter 6

Conclusion

Advancements in ML and DL have led to the development of cutting-edge autonomous systems, which require vast volumes of accurately labeled data. This research focused on the development of an autonomous aircraft perception system that accurately detected and labeled line markings on an airport taxiway, using a generalized framework called ALINA. The system employed an interactive trapezoidal ROI, geometrical modification, color space transformation, and histogram analysis techniques. The CIRCLEDAT algorithm is a vital component of ALINA that was used for detecting the pixels corresponding to line markings. ALINA has labeled 60,249 images from the taxiway dataset accurately and demonstrated its robustness in handling complex environmental settings, changeable weather, and atmospheric conditions. Specifically, when tested with 120 ground truth images consisting of different scenarios of the airport taxiway, ALINA achieved a detection rate of 0.9844. The potential of ALINA extends beyond the scope of this research and can be applied to other datasets. Future work could involve the integration of ALINA into autonomous taxiing systems to improve safety and efficiency in airports, thereby reducing the risks associated with human error. As computer vision techniques continue to advance, the development of autonomous perception systems like ALINA has the potential to revolutionize various industries by enabling accurate and efficient analysis of complex visual data.

Bibliography

- [1] *Accident statistics*. URL: <https://www.planecrashinfo.com/cause.htm> (visited on 04/10/2023).
- [2] *Airport Moving Maps - Jeppesen*. URL: <https://ww2.jeppesen.com/navigation-solutions/airport-moving-maps/> (visited on 04/10/2023).
- [3] Mihail-Alexandru Andrei et al. “Robust Lane Detection and Tracking Algorithm for Steering Assist Systems”. In: *Machines* 10.1 (Jan. 2022). Number: 1 Publisher: Multidisciplinary Digital Publishing Institute, p. 10. ISSN: 2075-1702. DOI: 10.3390/machines10010010. URL: <https://www.mdpi.com/2075-1702/10/1/10> (visited on 03/28/2023).
- [4] Sana K. Ardestani. *Combination of Texture, Color and Shape Operators to Describe Image Content: A Survey*. Dec. 18, 2020. DOI: 10.20944/preprints202012.0479.v1. URL: <https://www.preprints.org/manuscript/202012.0479/v1> (visited on 03/28/2023).
- [5] Aman Batra and Jason Gauci. “Aerodrome Taxiway Line Detection and Cross-Track Error Estimation using Computer Vision Techniques”. In: *2020 IEEE Aerospace Conference*. 2020 IEEE Aerospace Conference. ISSN: 1095-323X. Mar. 2020, pp. 1–9. DOI: 10.1109/AERO47225.2020.9172787.
- [6] Nasim Sepehri Boroujeni, S. Ali Etemad, and Anthony Whitehead. “Robust Horizon Detection Using Segmentation for UAV Applications”. In: *2012 Ninth Conference on Computer and Robot Vision*. 2012 Ninth Conference on Computer and Robot Vision. May 2012, pp. 346–352. DOI: 10.1109/CRV.2012.52.
- [7] Gary Bradski and Adrian Kaehler. *Learning OpenCV: Computer Vision with the OpenCV Library*. Google-Books-ID: seAgiOfu2EIC. ”O’Reilly Media, Inc.”, Sept. 24, 2008. 579 pp. ISBN: 978-0-596-55404-0.
- [8] Wilhelm Burger and Mark J. Burge. “Scale-Invariant Feature Transform (SIFT)”. In: *Digital Image Processing: An Algorithmic Introduction*. Ed. by Wilhelm Burger and Mark J. Burge. Texts in Computer Science. Cham: Springer International Publishing, 2022, pp. 709–763. ISBN: 978-3-031-05744-1. DOI: 10.1007/978-3-031-05744-1_25. URL: https://doi.org/10.1007/978-3-031-05744-1_25 (visited on 03/28/2023).

- [9] Tairui Chen et al. “Road marking detection and classification using machine learning algorithms”. In: *2015 IEEE Intelligent Vehicles Symposium (IV)*. 2015 IEEE Intelligent Vehicles Symposium (IV). ISSN: 1931-0587. June 2015, pp. 617–621. DOI: 10.1109/IVS.2015.7225753.
- [10] *Computer Vision*. Guide books. Archive Location: world. DOI: 10.5555/580035. URL: <https://dl.acm.org/doi/abs/10.5555/580035> (visited on 04/10/2023).
- [11] Ling Ding et al. “A comprehensive approach for road marking detection and recognition”. In: *Multimedia Tools and Applications* 79.23 (June 1, 2020), pp. 17193–17210. ISSN: 1573-7721. DOI: 10.1007/s11042-019-08384-5. URL: <https://doi.org/10.1007/s11042-019-08384-5> (visited on 03/28/2023).
- [12] *Evaluation of a Prototype Advanced Taxiway Guidance System (ATGS)*. AirportTech. URL: <https://www.airporttech.tc.faa.gov/Products/Airport-Safety-Papers-Publications/Airport-Safety-Detail/evaluation-of-a-prototype-advanced-taxiway-guidance-system-atgs> (visited on 04/10/2023).
- [13] Sergiy Fefilatyev et al. “Horizon Detection Using Machine Learning Techniques”. In: *2006 5th International Conference on Machine Learning and Applications (ICMLA'06)*. 2006 5th International Conference on Machine Learning and Applications (ICMLA'06). Dec. 2006, pp. 17–21. DOI: 10.1109/ICMLA.2006.25.
- [14] Haiyan Guan et al. “Road marking extraction in UAV imagery using attentive capsule feature pyramid network”. In: *International Journal of Applied Earth Observation and Geoinformation* 107 (Mar. 1, 2022), p. 102677. ISSN: 1569-8432. DOI: 10.1016/j.jag.2022.102677. URL: <https://www.sciencedirect.com/science/article/pii/S0303243422000034> (visited on 03/28/2023).
- [15] Any Gupta and Ayesha Choudhary. “A Framework for Camera-Based Real-Time Lane and Road Surface Marking Detection and Recognition”. In: *IEEE Transactions on Intelligent Vehicles* 3.4 (Dec. 2018). Conference Name: IEEE Transactions on Intelligent Vehicles, pp. 476–485. ISSN: 2379-8904. DOI: 10.1109/TIV.2018.2873902.
- [16] Toan Minh Hoang, Se Hyun Nam, and Kang Ryoung Park. “Enhanced Detection and Recognition of Road Markings Based on Adaptive Region of Interest and Deep Learning”. In: *IEEE Access* 7 (2019). Conference Name: IEEE Access, pp. 109817–109832. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2019.2933598.
- [17] Xinyu Jiao et al. “Real-time lane detection and tracking for autonomous vehicle applications”. In: *Proceedings of the Institution of Mechanical Engineers, Part D: Journal of Automobile Engineering* 233.9 (Aug. 1, 2019). Publisher: IMECHE, pp. 2301–2311. ISSN: 0954-4070. DOI: 10.1177/0954407019866989. URL: <https://doi.org/10.1177/0954407019866989> (visited on 03/28/2023).

- [18] Jeffrey Kiske and Serra Mall. “Automatic Labeling of Lane Markings for Autonomous Vehicles”. In: ().
- [19] *Lane detection technique based on perspective transformation and histogram analysis for self-driving cars* | Elsevier Enhanced Reader. DOI: 10.1016/j.compeleceng.2020.106653.
- [20] Zuo-Quan Li, Hui-Min Ma, and Zheng-Yu Liu. “Road Lane Detection with Gabor Filters”. In: *2016 International Conference on Information System and Artificial Intelligence (ISAI)*. 2016 International Conference on Information System and Artificial Intelligence (ISAI). June 2016, pp. 436–440. DOI: 10.1109/ISAI.2016.0099.
- [21] Rohit Maurya, P.R Gupta, and Ajay Shankar Shukla. “Road extraction using K-Means clustering and morphological operations”. In: *2011 International Conference on Image Information Processing*. 2011 International Conference on Image Information Processing. Nov. 2011, pp. 1–6. DOI: 10.1109/ICIIP.2011.6108839.
- [22] Claire Meymandi-Nejad et al. “Aircraft Navigation on Taxiways: Evaluation of Line Detection Algorithms Proposed for Automotive Applications”. In: *Proceedings of the 2020 4th International Symposium on Computer Science and Intelligent Control*. ISCSIC 2020: 2020 4th International Symposium on Computer Science and Intelligent Control. Newcastle upon Tyne United Kingdom: ACM, Nov. 17, 2020, pp. 1–6. ISBN: 978-1-4503-8889-4. DOI: 10.1145/3440084.3441202. URL: <https://dl.acm.org/doi/10.1145/3440084.3441202> (visited on 03/28/2023).
- [23] *New image descriptors based on color, texture, shape, and wavelets for object and scene image classification* | Elsevier Enhanced Reader. DOI: 10.1016/j.neucom.2013.02.014.
- [24] Md. Abdullah Noman et al. “A computer vision-based lane detection technique using gradient threshold and hue-lightness-saturation value for an autonomous vehicle”. In: *International Journal of Electrical and Computer Engineering* 13 (Feb. 1, 2023), pp. 347–357. DOI: 10.11591/ijece.v13i1.pp347-357.
- [25] Ali PourYazdanpanah et al. “Real-Time Horizon Line Detection based on Fusion of Classification and Clustering”. In: *International Journal of Computer Applications* 121.10 (July 18, 2015), pp. 5–11. ISSN: 09758887. DOI: 10.5120/21574-4625. URL: <http://research.ijcaonline.org/volume121/number10/pxc3904625.pdf> (visited on 03/28/2023).
- [26] Li Shuhua and Guo Gaizhi. “The application of improved HSV color space model in image processing”. In: *2010 2nd International Conference on Future Computer and Communication*. 2010 2nd International Conference on Future Computer and Communication. Vol. 2. May 2010, pp. V2–10–V2–13. DOI: 10.1109/ICFCC.2010.5497299.

- [27] P. Stern. “ADVANCED SURFACE MOVEMENT GUIDANCE SYSTEM: INSTALLING HONEYWELL’S A- SMGCS SYSTEM HAS HELPED REDUCE GROUND DELAYS AT INCHEON INTERNATIONAL AIRPORT (IIA)”. In: *Airports International* 35.7 (Sept. 2001). ISSN: 0002-2853. URL: <https://trid.trb.org/view/694274> (visited on 04/10/2023).
- [28] T. Tateyama et al. “Segmentation of high resolution satellite images by direction and morphological filters”. In: *Fourth International Conference on Hybrid Intelligent Systems (HIS’04)*. Fourth International Conference on Hybrid Intelligent Systems (HIS’04). Dec. 2004, pp. 482–487. DOI: 10.1109/ICHIS.2004.85.
- [29] Kevin Theuma and David Zammit Mangion. “An Image Processing Algorithm for Ground Navigation of Aircraft”. In: *Advances in Aerospace Guidance, Navigation and Control*. Ed. by Joël Bordeneuve-Guibé, Antoine Drouin, and Clément Roos. Cham: Springer International Publishing, 2015, pp. 381–399. ISBN: 978-3-319-17518-8. DOI: 10.1007/978-3-319-17518-8_22.
- [30] Kevin Theuma et al. “A Particle Filter for Ground Obstacle Tracking in the Airfield”. In: *AIAA Scitech 2019 Forum*. AIAA Scitech 2019 Forum. San Diego, California: American Institute of Aeronautics and Astronautics, Jan. 7, 2019. ISBN: 978-1-62410-578-4. DOI: 10.2514/6.2019-1265. URL: <https://arc.aiaa.org/doi/10.2514/6.2019-1265> (visited on 04/10/2023).
- [31] *Towards Practical Verification of Machine Learning: The Case of Computer Vision Systems*. DOI: 10.48550/arXiv.1712.01785. URL: <http://arxiv.org/abs/1712.01785>.
- [32] Joseph N. Wilson and Gerhard X. Ritter. *Handbook of Computer Vision Algorithms in Image Algebra*. Google-Books-ID: JYTLBQAAQBAJ. CRC Press, Sept. 21, 2000. 444 pp. ISBN: 978-1-4200-4238-2.